CADT IDT

W2 PRACTICE

Native HTTP and Manual Routing

At the end of this practice, you can

- ✓ **Create** and run a native Node.js HTTP server
- ✓ **Manually implement** route handling using conditionals.

- ✓ Serve **static files** using fs.
- ✓ Parse form data from POST requests.
- ✓ Debug and enhance server code using console outputs.

Get ready before this practice!

- ✓ **Read** the following documents to understand Nodejs built-in HTTP module: https://nodejs.org/api/http.html

- ✓ **Read** the following documents to understand Anatomy of an HTTP Transaction: https://nodejs.org/en/learn/modules/anatomy-of-an-http-transaction

How to submit this practice?

- ✓ Once finished, push your **code to GITHUB**
- ✓ Join the **URL of your GITHUB** repository on LMS



Lab2.1: *The "Ping" Server*

Goal: Learn how to initialize a server and verify it's alive.

```
const http = require('http');

const server = http.createServer((req, res) => {
  console.log('A request was received!');
  res.end('Pong');
});

server.listen(3000, () => {
  console.log('Warm-up 1 running on http://localhost:3000');
});
```

Task:

1. Run the code.
2. Open your browser to `http://localhost:3000`.

Lab2.2 *The Modern Toolkit (Apidog Setup)*

Goal: Install Apidog and use it to "talk" to your Node.js server. This replaces the need for using just the browser or complex `curl` commands.

Task 1: Installation

1. **Download:** Go to apidog.com/download and download the desktop client for your OS (Windows, Mac, or Linux).
    - *Tip:* While there is a web version, the **Desktop Client** is better for beginners because it can easily access `localhost` without extra browser security headaches.
2. **Open & Guest Mode:** You don't necessarily need an account to start. You can use it as a "Guest" or sign up to save your progress.

Task 2: The First Handshake

1. **Run your server:** Use the code from **Warm-Up 1** (the "Ping" server).
2. **Create a Request in Apidog:**
    - Click the **"+"** button to create a new tab.
    - Set the method to `GET`.
    - Enter the URL: `http://localhost:3000`.
    - Click **Send**.
3. **Inspect:** Look at the **Body** tab in the response section at the bottom. Does it say "Pong"?

Lab2.3 *Inspecting the Guest (The req Object)*

Goal: Understand that every request carries information about where it came from.

```
const http = require('http');

const server = http.createServer((req, res) => {
  // Your code here: Log the req.url and req.method

  res.end('Check your terminal to see the request details.');
});

server.listen(3000);
```

Task:

1. Inside the function, use `console.log()` to print `req.url` and `req.method`.
2. Visit `http://localhost:3000/hello-world`.
3. Observe the terminal. What does the `req.url` show?

Lab2.4 *Setting the Vibe (Status Codes)*

Goal: Learn how to tell the browser if the request was successful or not using HTTP status codes.

```
const http = require('http');

const server = http.createServer((req, res) => {
  // We want to tell the browser "Everything is OK" (200)
  // or "Not Found" (404)
  res.statusCode = 200;
  res.end('Status Code is set to 200');
});

server.listen(3000);
```

Task:

1. Change the `res.statusCode` to `404`.
2. Refresh your browser.
3. Open the **Browser Developer Tools** (F12) -> **Network Tab**. Refresh again and click on the "localhost" request. What color is the status code?

Lab2.5 *Text vs. HTML (Response Headers)*

Goal: Use headers to tell the browser how to interpret the data you send.

```
const http = require('http');

const server = http.createServer((req, res) => {
  // Task: Switch between 'text/plain' and 'text/html'
  res.setHeader('Content-Type', 'text/plain');
  res.write('<h1>Hello!</h1><p>Is this rendered as HTML?</p>');
  res.end();
});

server.listen(3000);
```

Task:

1. Run the code as-is. Does the browser show the `<h1>` tags or a big heading?
2. Change `'text/plain'` to `'text/html'`.
3. Refresh the browser and observe the difference.

Lab2.6 *The Live Clock (Dynamic Content)*

Goal: Realize that Node.js can execute logic *before* sending the response.

```
const http = require('http');

const server = http.createServer((req, res) => {
  const currentTime = new Date().toLocaleTimeString();

  res.setHeader('Content-Type', 'text/html');
  res.write(`<h1>Server Time</h1>`);
  res.write(`<p>The current time is: ${currentTime}</p>`);
  res.end();
});

server.listen(3000);
```

Task:

1. Run the server and visit the page.
2. Refresh the page 5 seconds later.
3. **Reflect:** Why does the time change even though you didn't change the code? This is the power of a dynamic server vs. a static HTML file!

Lab2.7 *ANALYZE*

Goal

- ✓ Identify and fix the bug.
- ✓ Understand the request-response cycle in Node.js using the `http` module.
- ✓ Explain the role of `res.write()` and `res.end()` in sending data back to the client.

For this exercise, you are provided with a minimal `server.js` file. Read and run the code and observe how it behaves.

```
// server.js
const http = require('http');

const server = http.createServer((req, res) => {
  res.write('Hello, World!');
  return res.endd();
});

server.listen(3000, () => {
  console.log('Server running on http://localhost:3000');
});
```

Q1 – What error message do you see in the terminal when you access `http://localhost:3000`? What line of code causes it?

Q2 – What is the purpose of `res.write()` and how is it different from `res.end()`?

Q3 – What do you think will happen if `res.end()` is not called at all?

Q4 – Why do we use `http.createServer()` instead of just calling a function directly?

Q5 – How can the server be made more resilient to such errors during development?

Lab2.8 *MANIPULATE (START CODE EX2)*

Goal

- ✓ Practice using `req.url` and `req.method`.
- ✓ Understand how manual routing mimics what frameworks (like Express) automate.
- ✓ Serve both plain text and raw HTML manually.

- For this exercise you will start with a **START CODE (EX-2)**

TASK 1 - Update the code above to add custom responses for these routes:

ROUTE	HTTP METHOD	RESPONSE
`/about`	**GET**	`About us: at CADT, we love node.js!`
`/contact-us`	**GET**	`You can reach us vai email…`
`/products`	**GET**	`Buy one get one…`
`/projects`	**GET**	`Here are our awesome projects`

Use VS Code's Thunder Client (or other tools (POSTMAN, INSOMIA) of your choice or curl on your terminal to make request.

Example output

```
curl http://localhost:3000/about ------------------------->
About us: at CADT, we love node.js!

curl http://localhost:3000/contact-us ------------------------
-> You can reach us vai email…
```

TASK 2 – As we can see the complexitiy grow as we add more routes. Use `switch` statement to arrange the code into more organized structure.

? Reflective Questions

1. What happens when you visit a URL that doesn't match any of the three defined?
2. Why do we check both the `req.url` and `req.method`?
3. What MIME type (`Content-Type`) do you set when returning HTML instead of plain text?
4. How might this routing logic become harder to manage as routes grow?
5. What benefits might a framework offer to simplify this logic?

Lab2.9 *CREATE (START CODE EX3)*

Goal

- ✓ Practice handling `POST` requests.
- ✓ Parse URL-encoded form data manually.
- ✓ Write and append to local files using Node.js' `fs` module.
- ✓ Handle async operations and errors gracefully.

- For this exercise you will start with a **START CODE EX-3**

TASK 1 - Extend your Node.js HTTP server to handle a **POST request** submitted from the contact form. When a user submits their name, the server should:

1. **Capture the form data** (from the request body).
2. **Log it to the console**.
3. **Write it to a local file** named `submissions.txt`.

```
Testing, go to /contact on broswer and test
```

Requirements

- Handle `POST /contact` requests.
- Parse raw `application/x-www-form-urlencoded` data from the request body.
- Write the name to a new line in `submissions.txt`.
- Send a success response to the client (HTML or plain text).

? Discussion Questions

1. Why do we listen for `data` and `end` events when handling `POST`?
2. What would happen if we didn't buffer the body correctly?
3. What is the format of form submissions when using the default browser form POST?
4. Why do we use `fs.appendFile` instead of `fs.writeFile`?
5. How could this be improved or made more secure?

Bonus Challenge (Optional)

- Validate that the `name` field is not empty before saving.
- Send back a small confirmation HTML page instead of plain text.
- Try saving submissions in JSON format instead of plain text.